



บททวน

3.1 The List

3.2 Array List

3.3 Linked list

1. Insert

กรณีไม่มีข้อมูล

- insert ได้เลย

กรณีมีข้อมูล

- insert หน้าสุด
- insert ตรงกลาง
- insert ท้าย

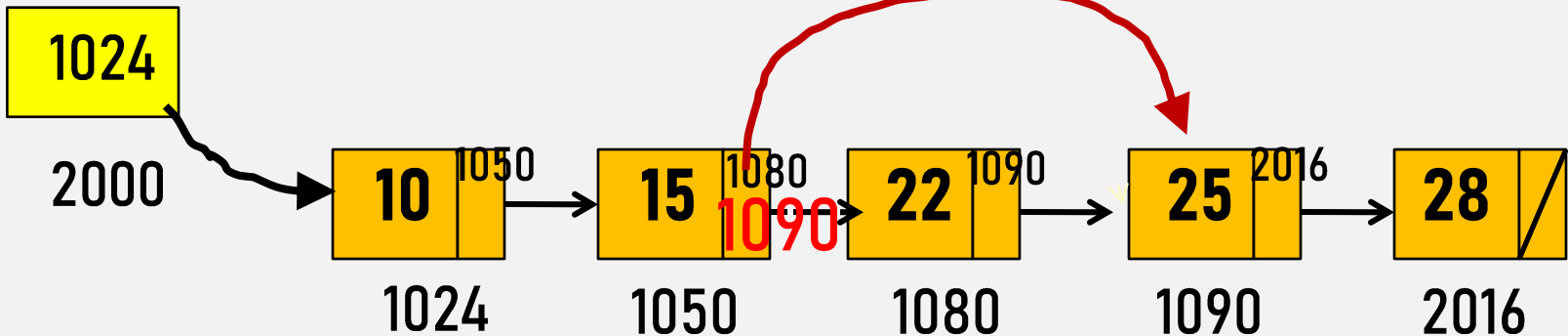
2. Print

3. Delete



Delete

Head



1. กรณีที่มี node เดียว
2. กรณีที่ลบ node แรก
3. กรณีลบตั้งแต่โหนดที่ 2 เป็นต้นไป



03603212 : Module2–List, stack, Queue 3

```
struct record *delete(struct record *head,int data)
```

```
{ struct record *node,*tmpfree;
```

```
node = head;
```

```
while(node)
```

```
{ if( data == node->next->value )
```

```
{ tmpfree = node->next;
```

```
node->next = node->next->next;
```

```
delete(tmpfree);
```

```
break;
```

```
}
```

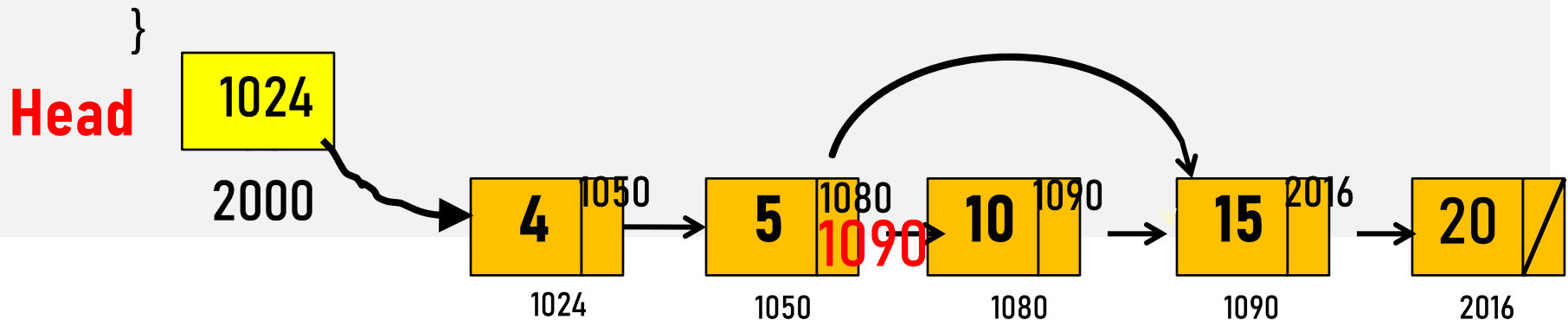
```
else
```

```
node=node->next;
```

```
} /* end while */
```

```
return node;
```

```
}
```





การบ้าน

1. Linked list

=====

MENU

=====

- 1) Insert
- 2) Delete
- 3) Print min to max, max to min
- 4) Count
- 5) Print first half and second half
- 6) Find

Please choose >

ทำการ insert โดยเลือกข้อ 1

Insert : 5

List = 5

จากนั้นให้กลับไปเมนู

ให้ทดลอง insert เพิ่มทีละตัวคือ 3 1 10 8 9

เลือกข้อ 3

: 1 3 5 8 9 10

: 10 9 8 5 3 1

จากนั้นถ้าเลือกข้อ 4 จะได้ค่า

Count = 6

ถ้าเลือกข้อ 5 จะได้ค่า

First = 1 3 5

Second = 8 9 10

ทำการลบโดยเลือกข้อ 2

Delete : 9

List : 1 3 5 8 10

จากนั้นกลับไปเมนู

ถ้าเลือกข้อ 5

First = 1 3 //หาร2=2

Second = 5 8 10

ถ้าเลือกข้อ 6

Find : 8

Found แต่ถ้าหาไม่เจอ จะพิมพ์ Not Found



การบ้าน

2. เก็บเลขจำนวนเต็มขนาดไม่จำกัด ให้นิสรับข้อมูลเป็นเลขจำนวนเต็มขนาดไม่จำกัด 2 ค่าใส่ linked list โดยมี p1 ชี้ตัวเลขตัวที่ 1 และ p2 ชี้ตัวเลขตัวที่ 2 (ตัวเลขนี้ใหญ่มากจนไม่สามารถเก็บลงในตัวแปร int, float, long double ได้ จึงต้องนำแต่ละ digit มาเก็บลง linked list

=====

MENU

=====

1. Input p1
 2. Input p2
 3. Add
 4. Exit
- Please choose >

ถ้าเลือกข้อ 1

Input : 899

P1 = 899

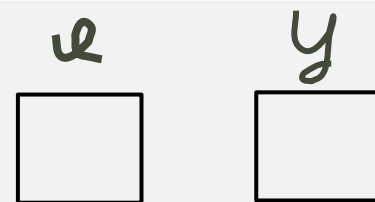
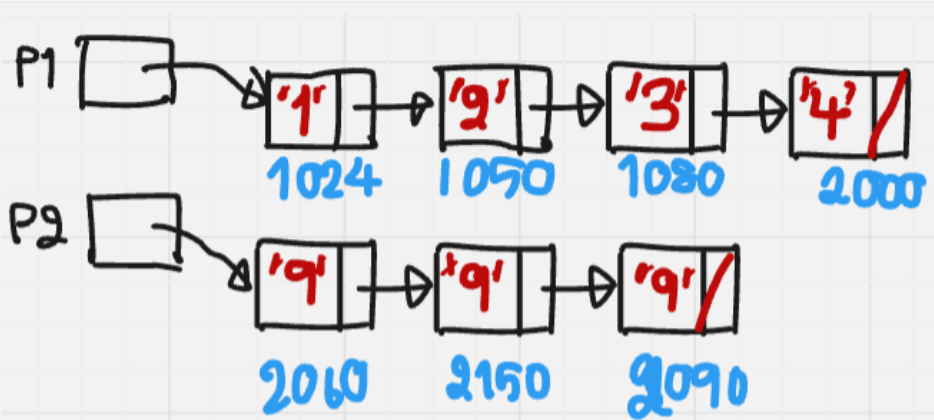
ถ้าเลือกข้อ 2

Input : 1234567890

P2 = 1234567890

ถ้าเลือกข้อ 3

Output = 1234568789



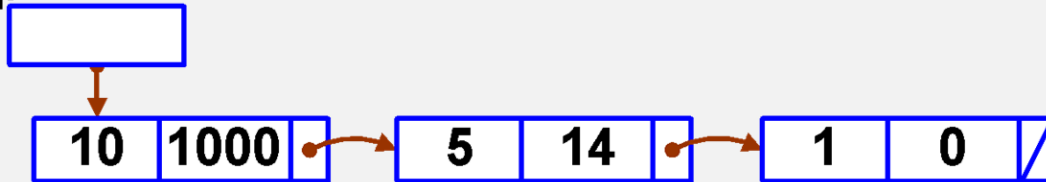
Examples

The polynomial ADT

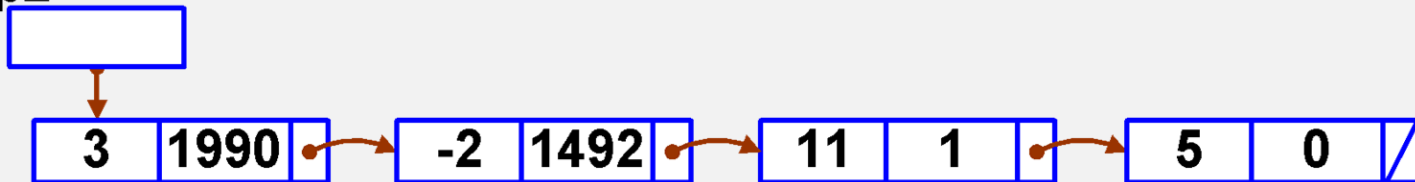
$$p1(x) = 10x^{1000} + 5x^{14} + 1$$

$$p2(x) = 3x^{1990} - 2x^{1492} + 11x + 5$$

p1



p2





การบ้าน สายลับข้ามชาติ

3. ให้นิสิต input passwd ที่เข้ารหัสแล้วใส่ใน list กำหนด structure ดังนี้

struct record

```
{ char c;  
  struct record *next;  
};
```

=====

MENU

=====

- 1) Input secret code
- 2) Decode
- 3) Exit

Please choose >

สมมติว่า รหัสคือ NOBOMB

รหัสจะทำการเลือกตัวอักษรคือออกมาก่อนคือ NBM จากนั้นจะเลือกตัวอักษรคู่ออกมา ต่อท้ายอักษรคือคือ OOB ดังนั้น

ข้อความ : NOBOMB

เข้ารหัส : NBMOOB

ถ้าเลือกข้อ 1 ให้ใส่รหัสลับที่นิสิตได้มา

Code : NBMOOB

จากนั้นกลับไปเมนู

ถ้าเลือกข้อ 2 ทำการถอดรหัส

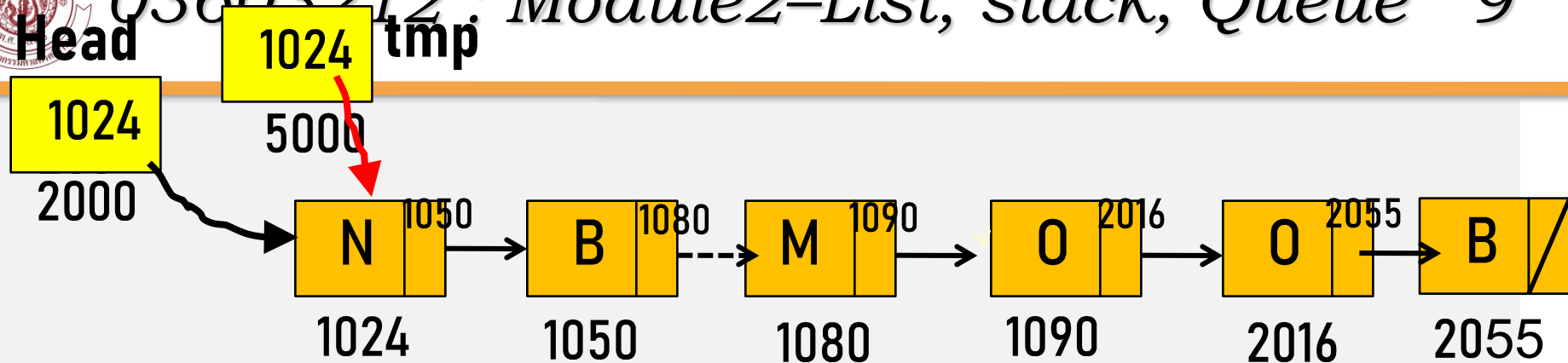
Answer : NOBOMB

หรือ

Code : KLIL

ถอดรหัส

Answer : KILL

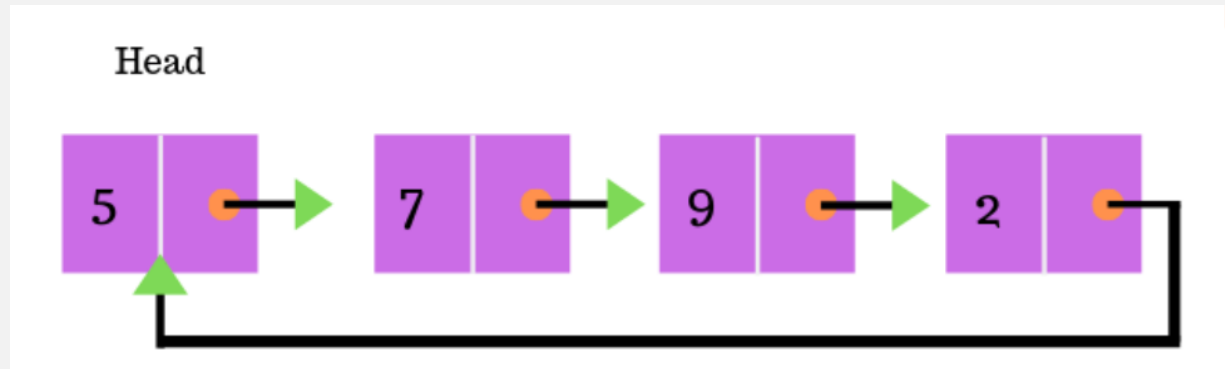


```
void print(struct spyrecord *head)
{
    cout << "\nSpy: \n";
    struct record *tmp;
    tmp=head;
    while(tmp!=NULL)
    {
        cout << tmp->value << " ";
        tmp=tmp->next->next;
    }
}
```

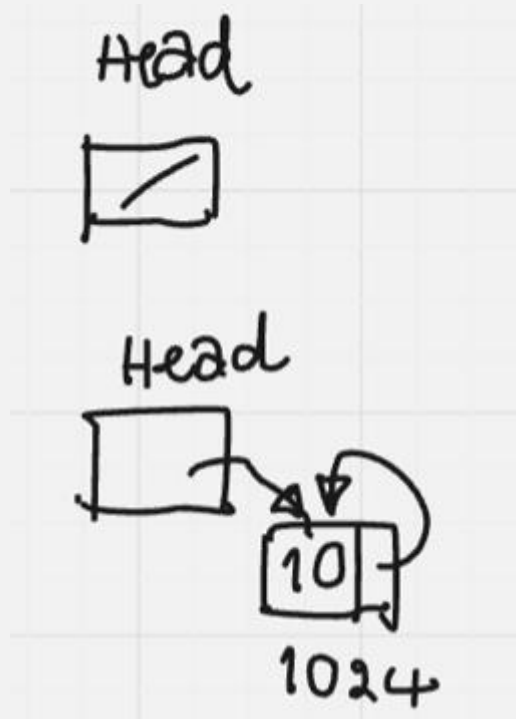
```
#include <iostream>
using namespace std;
int main()
{ char c;
  while(1)
  { c=getchar();
    if(c!='\n')
      cout << "Input : " << c << endl;
    else
      break;
  }
}
```

3.4 Circularly linked list

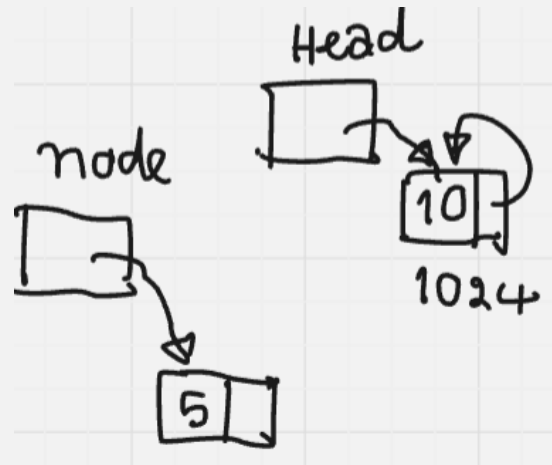
คือ linked list ชนิดหนึ่งที่แตกต่างกันจากการโยงปกติ ตรงที่โหนดสุดท้ายของรายการไม่ได้ชี้ไปยังค่า NULL แต่จะชี้กลับไปยังโหนดแรกของ list ทำให้เกิดลักษณะเป็นวงกลม



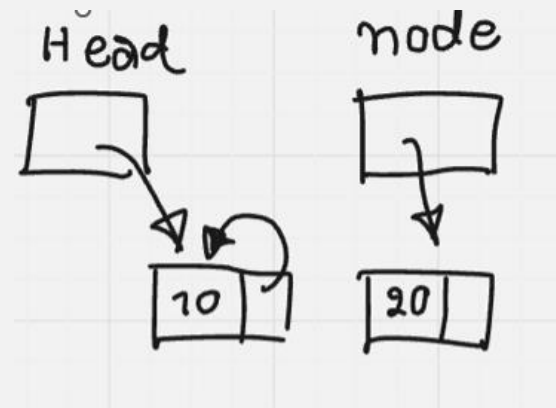
1) กรณีที่ไม่มีข้อมูล



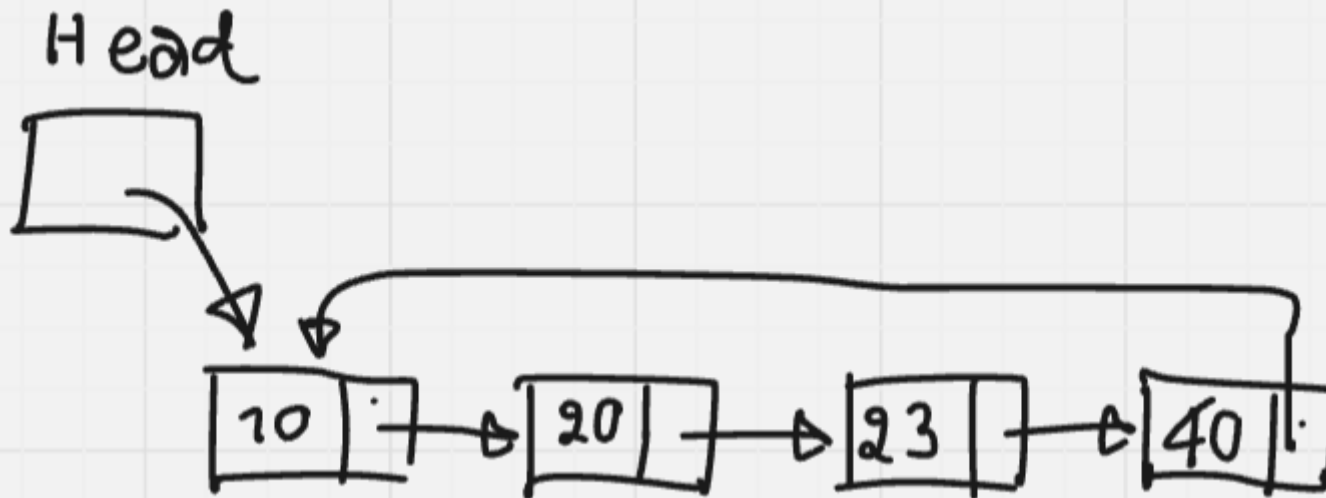
2) กรณี insert ด้านหน้า



3) กรณี insert ห้าย

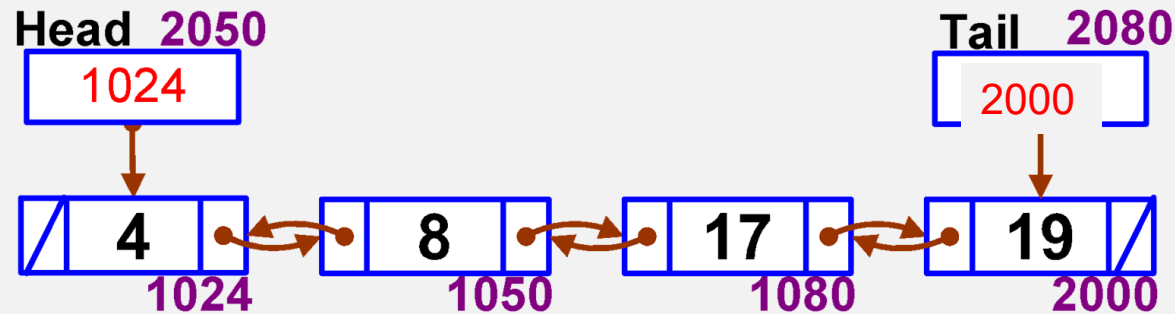


4) กรณี insert ตรงกลาง จะทำเหมือน linked list ปกติ



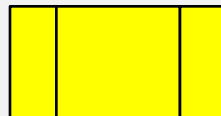
3.5 Doubly Linked Lists

The link list that add extra field to the data structure, containing a pointer to the previous cell.



struct record

```
{  int data;
   struct record *next;
   struct record *prev;
};
```



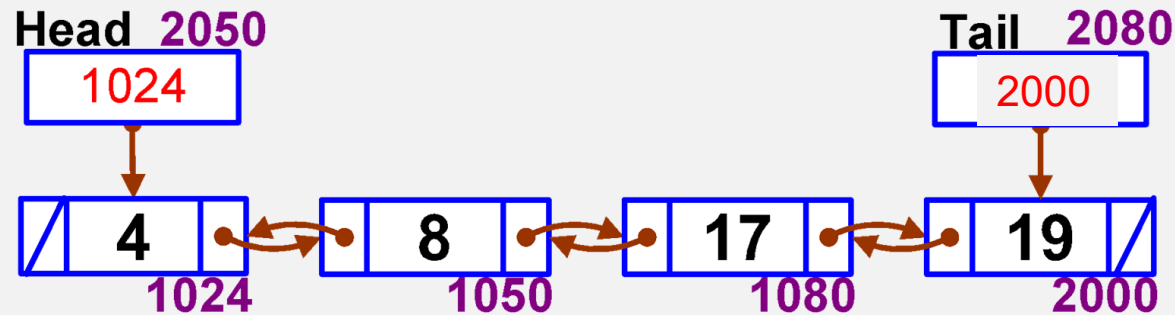
1) Insertion (Doubly Linklist)

1.1 Insert while no data in list

1.2 Insert first

1.3 Insert last

1.4 Insert middle



โครงโปรแกรมในการ insert

```

if(ยังไม่มีข้อมูล ?)
{
    ....
}
else
{
    สร้าง node เตรียมไว้
    if ( insert ด้านหน้า ? )
    {
        .....
    } else if( insert ด้านหลัง ? )
    {
        ....
    }
    else //ตรงกลาง
    {
        หาตำแหน่ง
        .....
    }
}

```

```

if(head==NULL)
{
    ....
}
else
{
    สร้าง node เตรียมไว้
    if( data .... head->value?)
    {
        .....
    } else if(
        )
    {
        ....
    }
    else
    {
        หาตำแหน่ง
        .....
    }
}

```



.....

```
struct record
{
    int value;
    struct record *prev;
    struct record *next;
};
struct record *tail=NULL;

struct record *insert(struct record *head,int data)
{
    if(head==NULL)
    {
        head=new struct record;
        head->value=data;
        head->next=head->prev=NULL;
        tail=head;
    }
    return head;
}
```



```
int main()
{
    struct record *head=NULL;

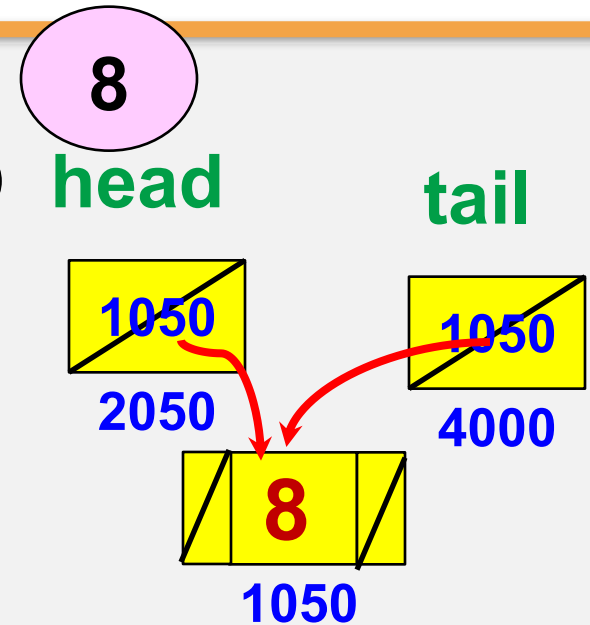
    head=insert(head,8);
    cout << head->value<< endl;
    cout << tail->value<< endl;
}
```

1.1 Insert while no data in list

```
struct record *insert(struct record *head, int data)
```

.... ประกาศตัวแปรเอง...

1. if(head == NULL)
2. { head=new struct rec;
3. head->value= data;
4. head->next=NULL;
5. head->prev=NULL;
6. tail=head;
7. }
8. return head;



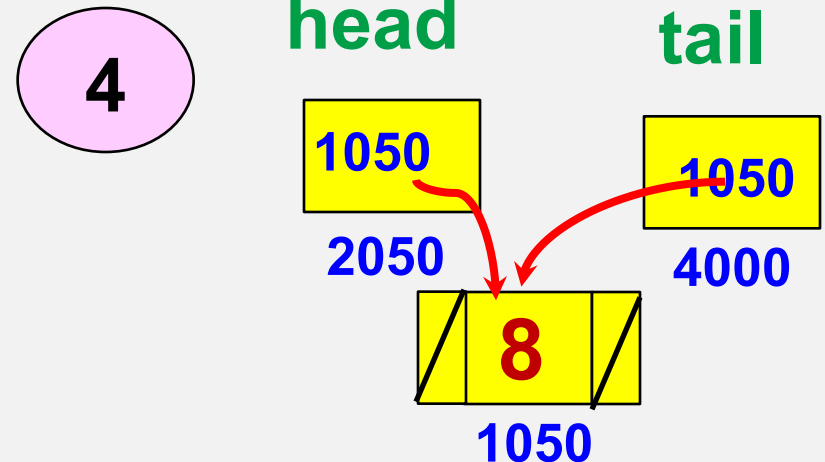


1.2. มีข้อมูลแล้วจะ Insert first

struct record *insert(struct record *head, int data)

.... ประกาศตัวแปรเอง...

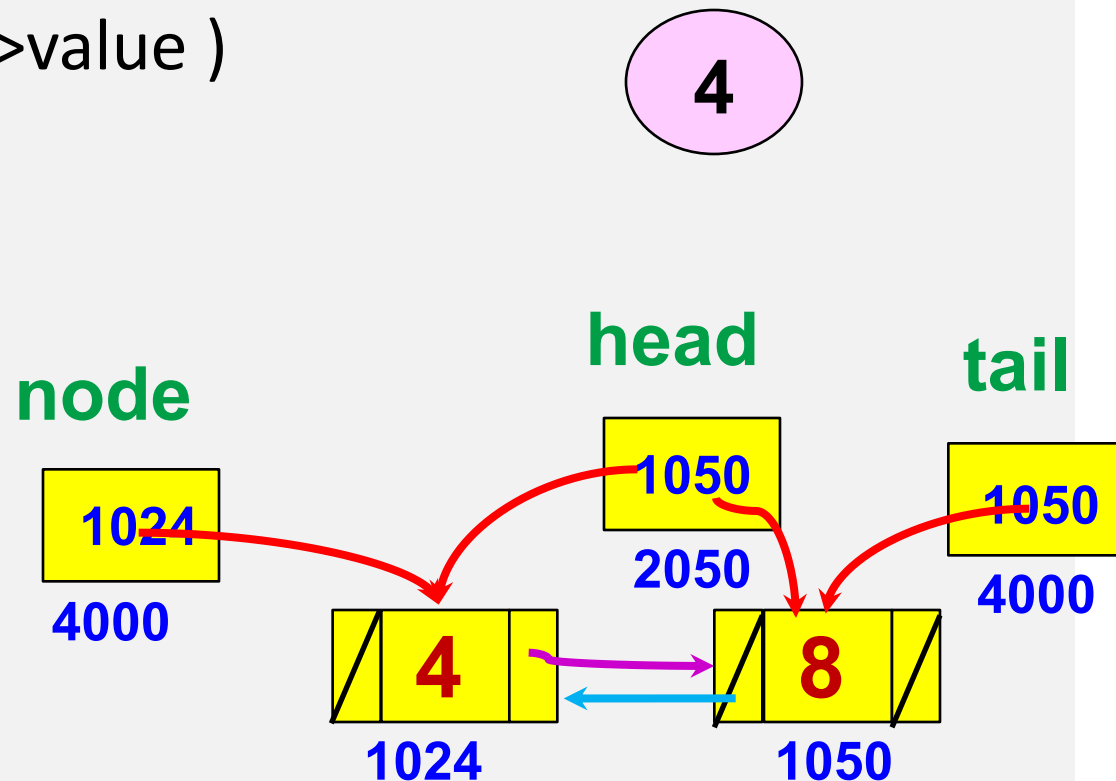
1. if(head == NULL)
2. { head=new struct rec;
3. head->value= data;
4. head->next=NULL;
5. head->prev=NULL;
6. tail=head;
7. }



8. else

9. { **node = สร้าง node เก็บข้อมูลใหม่**

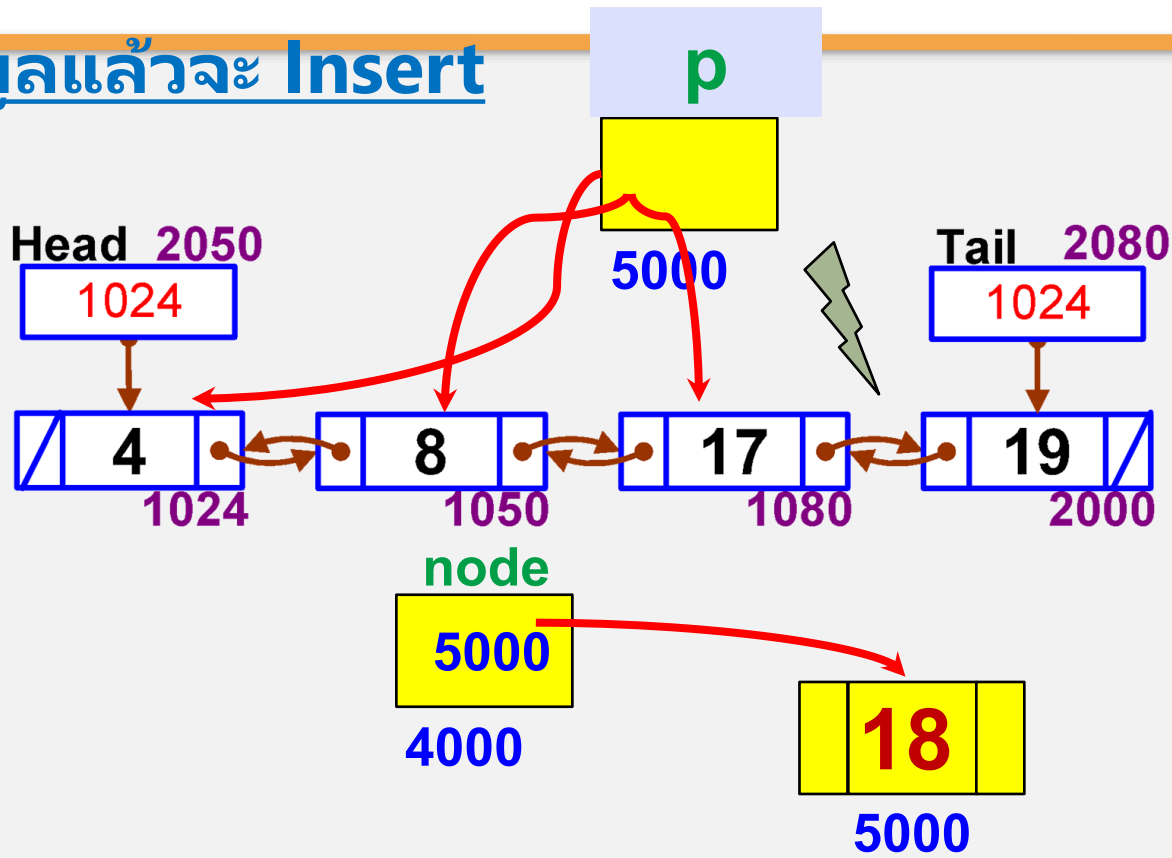
10. if (data <= head->value)
 {



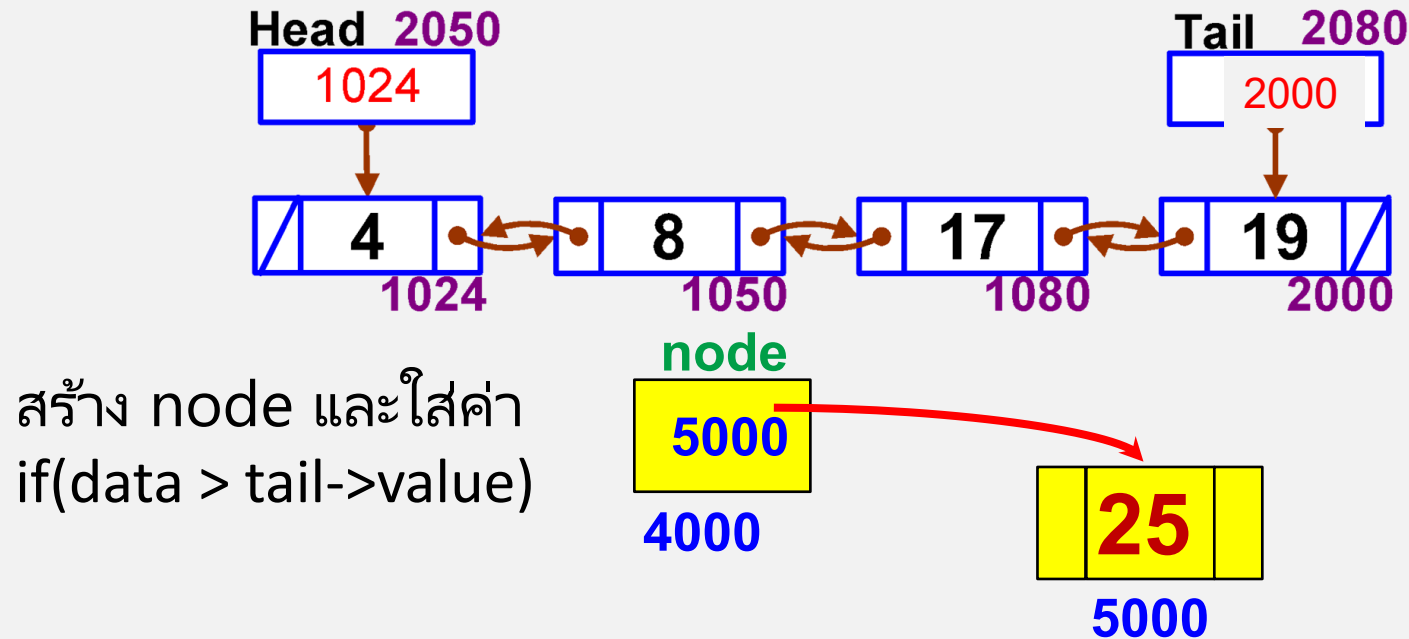
1.3. มีข้อมูลแล้วจะ Insert middle

18

else



1.4. มีข้อมูลแล้วจะ Insert last



25

การบ้าน

1. Doubly Linked list

=====

MENU

=====

1)Insert

2)Exit

Please choose >

ทำการ insert โดยเลือกข้อ 1

Insert : 5

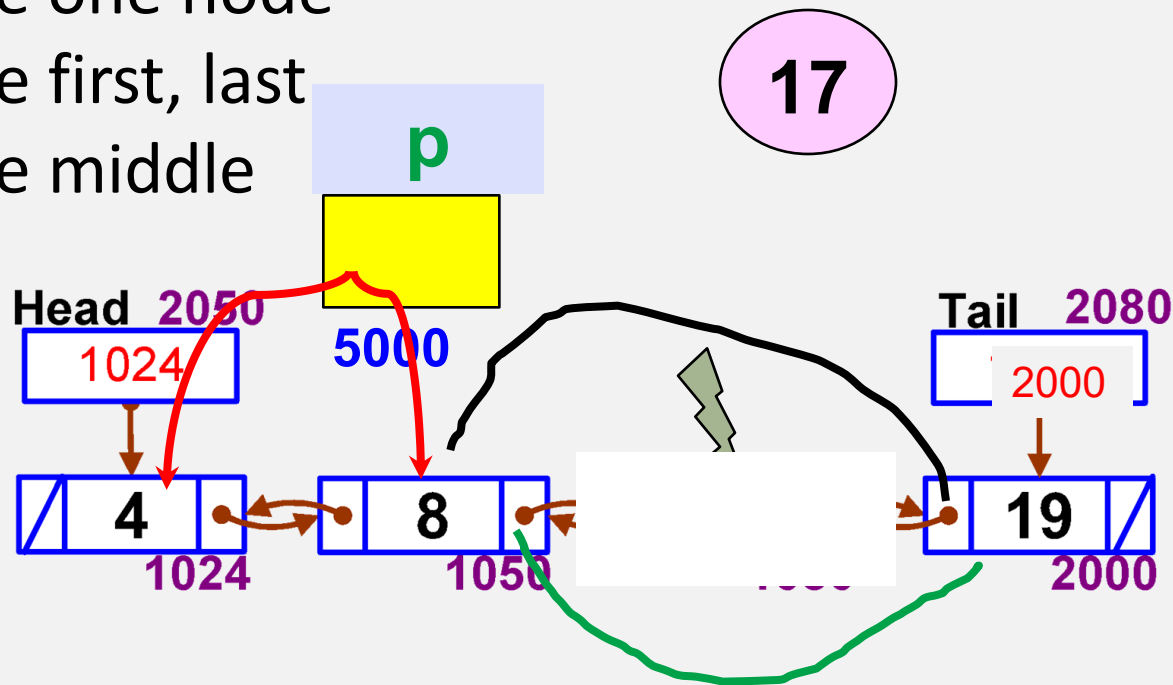
List = 5

จากนั้นให้กลับไปเมนู

ให้ทดลอง insert เพิ่มทีละตัวคือ 3 1 10 8 9

2) Delete (Doubly Linklist)

1. Delete while no data in list
2. Delete one node
3. Delete first, last
4. Delete middle



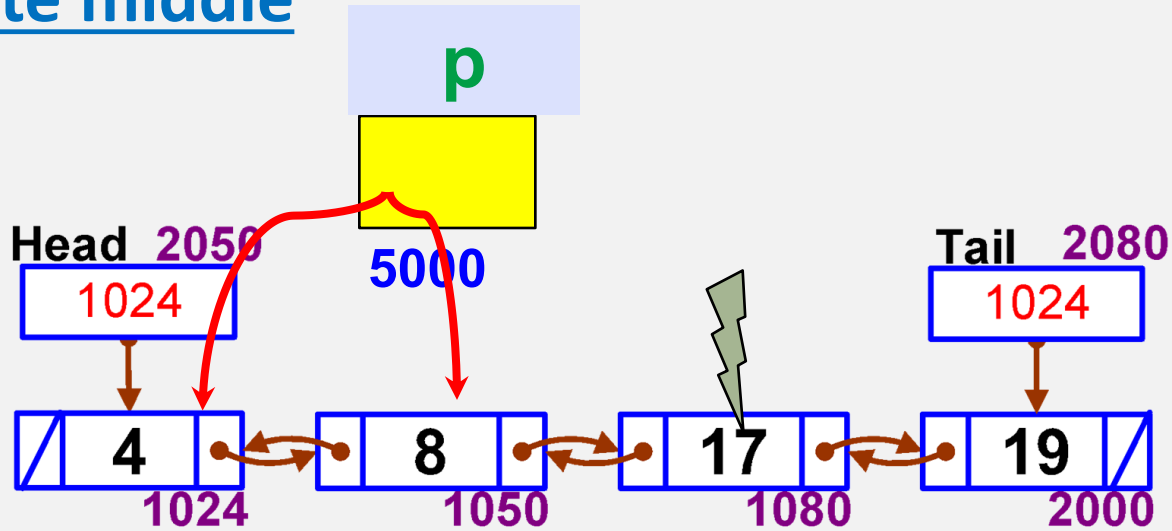
โครงโปรแกรมในการ delete

```
if(มี node เดียว ?)
{
    ....
}
else
{
    if ( ลบโหนดแรก ? )
    {
        .....
    } else if(ลบโหนดท้าย? )
    {
        ....
    }
    else //ตรงกลาง
    {
        หาตำแหน่ง
        .....
    }
}
```



Delete middle

17



3.6 Circularly doubly linked lists :

A popular convention is to have the last cell keep pointer back to the first. This can be done with or without header (If the header present, the last cell point to it.)

